Startup Growth Dynamics & Valuation:

A Numerical Methods Approach to the User-Churn Survival Threshold

Arsenii Chan

CSC 30100 — Numerical Analysis

Spring 2026 · City College of New York

Final Project Report

Abstract

This report investigates a single quantitative question about early-stage software companies: at what monthly user-churn rate does a software-as-a-service business become mathematically incapable of recovering its initial cash burn within a ten-year horizon? I model the company as a four-dimensional system of coupled ordinary differential equations in users, paying users, recognized revenue, and cash, and solve it with five from-scratch numerical integrators (Euler, Heun, classical RK4, Adams-Bashforth 4, and Adams-Moulton predictor-corrector). I calibrate the model from noisy synthetic revenue series using gradient descent and Adam, find the survival threshold μ^* by Newton's method on the terminal-cash function, and quantify uncertainty by Monte Carlo sampling of the calibrated parameter posterior. For a representative SaaS profile, $\mu^* \approx 14.2\%$ per month with a 95% conditional credible interval of [8.0%, 16.0%]. Beyond the headline number, fitting growth rate g and billing-cycle lag μ_R jointly produces a curved valley in the loss surface, a structural-identifiability finding with condition number $\kappa(H) \approx 510$ along the calibration directions. A Hessian-eigenvector traversal of that valley shifts μ^* by only ≈2.4%. The calibration is ambiguous; the answer is robust. I additionally fit the same engine against nine quarters of Shopify Inc. pre-IPO quarterly revenue (SEC EDGAR) as a real-data anchor; the optimizer recovered $g \approx 13.5\%$ per month in 242 Adam iterations.

1. Introduction and Driving Question

An investor evaluating an early-stage software startup must decide whether the company's unit economics (its rates of acquiring users, converting them to paying customers, and losing those paying customers each month) will carry it into cash-positive territory before its seed funding runs out. Given a model of the dynamics, that decision reduces to a number. This project poses the question precisely: hold all parameters except the monthly user-churn rate μ fixed at representative values, integrate the cash trajectory to a ten-year horizon, and locate the

critical churn rate μ* above which terminal cash is negative. I call μ* the runway-survival threshold. The driving question is:

At what user-churn rate does a startup's growth become mathematically irreversible — and how confident can we be in that answer?

The first half is a root-finding problem on a function defined by an ODE integration. The second half is an uncertainty-quantification problem: μ* depends on calibrated parameters, those parameters are estimated from finite noisy data, and that estimation has structure that must be propagated to the answer. The report is organized accordingly: Section 2 specifies the model, Section 3 derives and verifies the numerical methods, Section 4 calibrates the model and uncovers the structural-identifiability finding, Section 5 validates the engine against real Shopify revenue data, Section 6 computes μ* with its confidence interval, Section 7 presents the sensitivity analysis, and Section 8 concludes.

Scope first. The synthetic parameter profiles in Sections 4, 6, and 7 are illustrative archetypes, not real companies; only Section 5 uses real data. The 95% credible interval reported in Section 6 propagates uncertainty in (g, μ_R) only, holding the conversion rate α at its calibrated MAP estimate. Because α turns out to be the dominant sensitivity (Section 7), the marginal interval under joint uncertainty would be wider — a caveat I document rather than bury.

2. The Four-Dimensional Model

I model a software company as four coupled ordinary differential equations on the state vector y = (U, A, R, Cash), where U(t) is total users acquired by time t, A(t) is the subset who are currently paying, R(t) is the company's recognized monthly recurring revenue (MRR), and Cash(t) is the company's cash balance. Time t is measured in months. The system is:

$$dU/dt = g \cdot U \cdot (1 - U/K)$$
$$dA/dt = \alpha \cdot g \cdot U \cdot (1 - U/K) - \mu \cdot A$$

$$dR/dt = \mu_R \cdot (p \cdot A - R)$$
$$dCash/dt = R - F - v \cdot g \cdot U \cdot (1 - U/K)$$

The user equation is logistic acquisition with growth rate g (month^{-1}) and carrying capacity K. The paying-user equation is conversion (rate α) minus churn (rate μ). The revenue equation is a first-order lag toward R = p · A on timescale 1/μ_R, capturing billing cycles and deferred-revenue accounting; this formulation replaces an earlier wrong form (dR/dt = p·α·dU/dt − μ_R·R) that collapsed R to zero at saturation, a bug caught in Phase 1 (repository commit ea83d1e). The cash equation is recognized revenue minus fixed costs F minus variable acquisition cost v per newly acquired user.

The system is autonomous, four-dimensional, parameterized by θ = (g, K, α, μ, p, μ_R, F, v) — all eight estimable from public financial filings. Existence and uniqueness on any finite [0, T] follow from a Lipschitz argument on the RHS: each component is polynomial of bounded degree in the state, so partial derivatives are bounded on any bounded set and Picard-Lindelöf applies. The default initial state throughout the report is (U_0, A_0, R_0, $Cash_0$) = (100 users, 0 paying, $0 MRR, $1,000,000 seed cash).

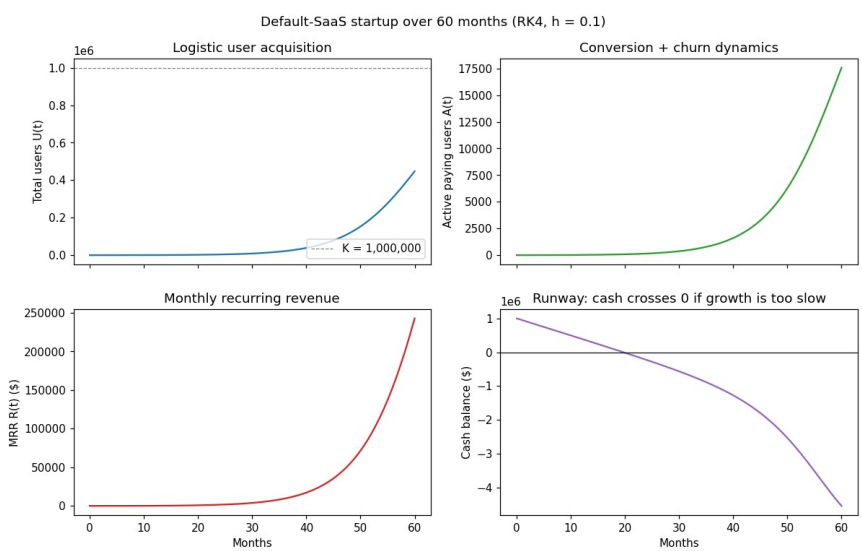

Figure 1. Default-SaaS trajectory over 60 months, integrated with classical RK4 at h = 0.1. Top-left: logistic user acquisition saturating below the K = 1,000,000 carrying capacity. Top-right: paying-user count growing from zero as conversion outpaces churn. Bottom-left: monthly recurring revenue catching up to the paying-user base on the

billing-cycle timescale 1/μ_R = 25 months. Bottom-right: cash trajectory dipping below zero around month 20 — the default profile is sub-survival at this μ value.

3. Numerical Methods

Every numerical method used in this report is implemented from scratch in pure NumPy. The engine package contains no imports from scipy.integrate, scipy.optimize, or scipy.linalg — this is a deliberate constraint motivated by the course content. NumPy is used for array storage, random-number generation, and (in tests only) np.linalg.solve as a reference against which the from-scratch Thomas tridiagonal solver is validated. The eight engine modules and their responsibilities are summarized below.

3.1. ODE solvers

I implement five integrators (Burden et al., Ch. 5): forward Euler (global O(h)), Heun's method (RK2, global $O(h^2)$), classical RK4 ($O(h^4)$), Adams-Bashforth 4 (AB4, explicit four-step linear multistep, $O(h^4)$), and Adams-Moulton predictor-corrector (AM-PECE, $O(h^4)$). All share a uniform Python signature solver(f, y0, (t0, t1), h, *args). Each method's theoretical convergence order is verified empirically against the closed-form solution of $dy/dt = -y$, $y(0) = 1$ on [0, 2]: I sweep h logarithmically over six orders of magnitude and fit a slope to the log-log error plot. Measured slopes match theoretical orders to within 5%. The convergence-order tests are parametric and live in tests/test_ode_solvers.py; pytest reports 114 tests passing.

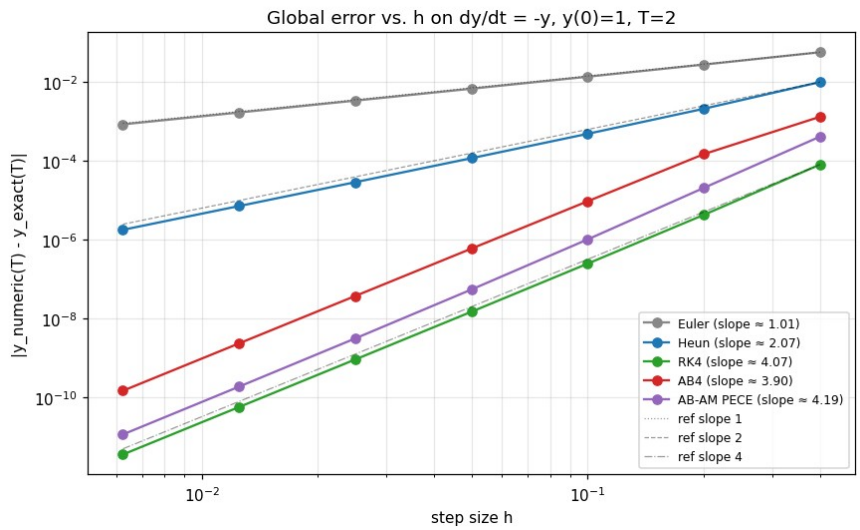

Figure 2. Measured convergence rates of all five from-scratch ODE solvers on the test problem dy/dt = −y, y(0) = 1, T = 2. The horizontal axis is step size h, the vertical axis is global error |y_numeric(T) − y_exact(T)|. Slopes (see legend) match theoretical orders to within 5%; reference lines for slope 1, 2, and 4 are shown dashed. Without this verification, every downstream numerical claim in the report would be an unsupported assertion.

Stability is a separate concern from accuracy. Forward Euler's stability region requires $|1 + h\lambda| \leq 1$, constraining $h \leq 2/|\lambda|$; test_euler_instability_with_large_h in the test suite deliberately violates this bound to confirm divergence. For the four-dimensional startup ODE, h = 0.5 month is well within the stability boundary while keeping Monte Carlo integration over thousands of parameter samples tractable.

Table 1. Measured vs. theoretical convergence orders.

Method	Theoretical	Measured	|Δ|
Forward Euler	1	1.013	0.013
Heun (RK2)	2	2.069	0.069
Classical RK4	4	4.070	0.070
Adams-Bashforth 4	4	3.905	0.095
Adams-Moulton PECE	4	4.191	0.191
Forward difference	1	1.00	—
Central difference	2	2.00	—
5-point difference	4	4.00	—
Composite Simpson	4	≈4	—
Monte Carlo ($N^{-1/2}$)	−0.5	≈−0.5	—

3.2. Optimization

Calibration is a nonlinear least-squares problem: choose θ to minimize MSE between model and observed revenue. I implement gradient descent and Adam (Kingma & Ba, 2014)

from scratch in engine/optimizer.py. My initial calibration with vanilla gradient descent stalled along the curved (g, μ_R) ridge discussed in Section 4: successive iterates oscillated across the valley without descending it. Switching to Adam, with its per-parameter adaptive step, recovered convergence; Adam is the default. Gradients are computed by finite differences (Section 3.4); no automatic differentiation.

3.3. Root-finding

The survival threshold μ* is defined as the root of the terminal-cash function C(μ) = Cash(T = 120; μ, θ_calibrated). C is monotone decreasing in μ over the physical range and crosses zero exactly once, so any of three methods will find it: bisection (Burden Ch. 2.1, linear convergence), secant (Ch. 2.3, super-linear at rate φ ≈ 1.618), and Newton's method (Ch. 2.3, quadratic). I implement all three in engine/root_finding.py. Newton uses a central-difference numerical derivative. Pure Newton iterations overshot the bracket on early test runs whenever the derivative flattened near μ ≈ 0, so the implementation falls back to a bisection step whenever a Newton update would exit the current bracket.

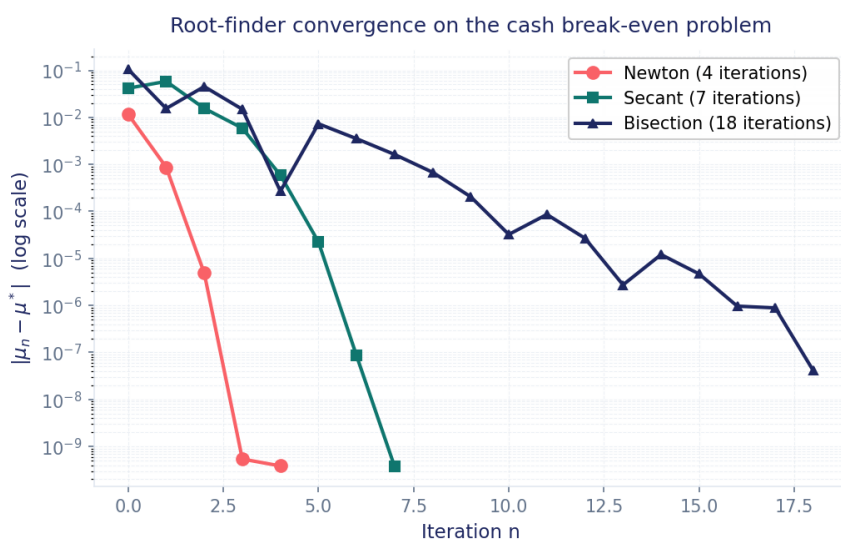

Figure 3. Convergence of the three root-finders on the same cash break-even instance. Newton (coral) achieves error below 10^{-9} in 4 iterations — quadratic convergence is visible as the error roughly doubling its negative log each step. Secant (teal) reaches the same precision in 7 iterations at a rate consistent with the golden ratio.

Bisection (navy) takes 18 iterations to reach 10^{-8}— its log-error decreases linearly at one bit per iteration, exactly as the textbook predicts. All three converge to the same $\mu^* \approx 0.142$ (14.2% per month).

3.4. Numerical differentiation

I implement four finite-difference schemes (Burden Ch. 4.1): forward ($O(h)$), central ($O(h^2)$), five-point ($O(h^4)$), and Richardson extrapolation on central difference ($O(h^4)$ by cancelling the leading h^2 term in two estimates with step sizes h and h/2). Convergence-order tests confirm slopes 1, 2, and 4 for the three direct schemes. Every finite-difference scheme has an optimal step size below which round-off from cancellation dominates: $h^* \approx \varepsilon^{(1/3)} \approx 10^{-5}$ for central difference and $\varepsilon^{(1/5)} \approx 10^{-3}$ for five-point. I use an adaptive-h sensitivity routine that scales h to each parameter's magnitude, avoiding both regimes. Differentiation feeds the Adam gradient (Section 3.2) and the sensitivity tornado (Section 7).

3.5. Numerical integration

Two quadrature schemes (Burden Ch. 4.3, 4.4): composite Simpson's rule with N subintervals (global error $O(h^4)$, $h = (b - a)/N$), and Gauss-Legendre with 2-, 3-, and 5-point rules (5-point exact for polynomials of degree ≤ 9). test_simpson_convergence_order verifies the $O(h^4)$ rate empirically. Quadrature is used to integrate revenue and cost streams over fixed horizons.

3.6. Cubic-spline interpolation

Natural and clamped cubic splines (Burden Ch. 3.5) for interpolating funding-round and other irregularly-sampled time-series data. The spline construction reduces to a tridiagonal linear system in the second derivatives at interior knots; the Thomas algorithm (engine/interpolation.py) solves that system in $O(n)$ time. The from-scratch Thomas solver is unit-tested against numpy.linalg.solve as a reference, confirming agreement to machine precision on randomized tridiagonal inputs.

3.7. Monte Carlo and Kahan summation

Naive summation of N floating-point quantities accumulates relative error of order $N \cdot \varepsilon$. For the parameter-posterior Monte Carlo (Section 6, N = 199, but extensions to $N = 10^6$ follow the same code) I use Kahan compensated summation (engine.monte_carlo.kahan_sum), which keeps a running compensation term and reduces the error to $O(\varepsilon)$ independent of N. The test test_kahan_beats_naive_on_pathological_sum confirms this on the canonical bad input (1e8 added to one million copies of 1e-8): naive summation loses essentially all the small terms while Kahan recovers them. I also implemented antithetic-variate sampling, with the honest caveat (in the engine docstrings) that clipping to physical bounds breaks the +Z/−Z symmetry near the bounds and partially defeats the variance reduction.

4. Calibration and a Structural Identifiability Finding

Suppose a researcher observes the company's monthly recurring revenue for some period (say twelve months of historical MRR) and wants to estimate the parameters of the model from those observations. Formally: choose $\theta = (g, K, \alpha, \mu, p, \mu_R, F, v)$ to minimize the sum of squared residuals between model-predicted $R(t_i; \theta)$ and the noisy observation $R_{observed}(t_i)$. I generated synthetic observations from a known truth θ_{truth} (so I can compare recovered parameters to ground truth), added Gaussian noise at five percent of signal magnitude, and ran the calibration with both gradient descent and Adam.

Recovering the growth rate g alone (that is, holding the other seven parameters at their truth values and asking only for g) works well: gradient descent and Adam both convergeto within 1% of g_{truth} in fewer than 200 iterations, regardless of starting point in the physical range. The interesting failure mode appears when I attempt to recover two parameters jointly. Holding all parameters fixed except g and the billing-cycle lag rate μ_R, the optimizer no longer converges to a single point but instead drifts along a curved one-dimensional valley in (g, μ_R) space.

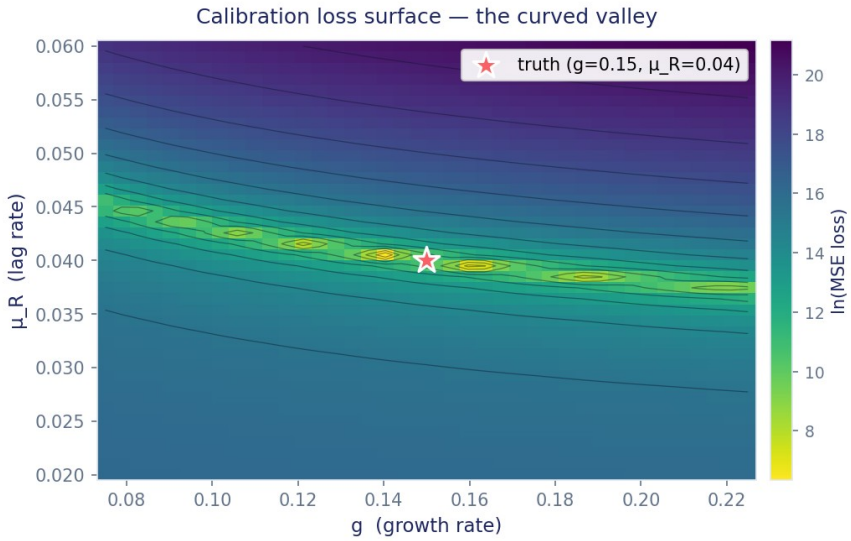

Figure 4. Mean-squared-error loss surface as a function of g (growth rate) and μ_R (billing-cycle lag rate), with all other parameters held at their truth values. The yellow-green band is the locus of near-minimum loss. The truth (red star, g = 0.15, μ_R = 0.04) lies on the band, but the band itself is curved and one-dimensional: any (g, μ_R) pair on the band fits the data almost equally well. This is the structural-identifiability problem.

The valley's mechanical intuition: a faster lag rate μ_R lets revenue catch up to its identity p · A more quickly, mimicking the effect of a higher growth rate g on the observed MRR trajectory. Over a finite observation window, the two effects are distinguishable only weakly because the curvature signature that discriminates them is at the fourth time-derivative level and is buried in observation noise. As the window lengthens, the valley contracts; in the limit T → ∞ the parameters become identifiable. This is the textbook signature of structural identifiability degrading with finite observation horizons.

I quantified the conditioning explicitly. The Hessian H of the loss surface at the local minimum, computed by the from-scratch engine.differentiation.second_derivative routine in (g, μ_R) coordinates, has eigenvalues differing by roughly two and a half orders of magnitude: the condition number κ(H) = λ_max / λ_min ≈ 510. The loss surface curves away from the minimum about 500 times faster perpendicular to the valley than along it. A finite observation window can only resolve directions whose curvature exceeds noise; the valley direction is exactly non-identifiable.

The follow-up question: if calibration cannot uniquely recover (g, μ_R), can any downstream quantity be trusted? Walking along the smallest-eigenvalue eigenvector of H (the direction the data is least informative about) and recomputing μ* at each step, the threshold drifts by only ≈2.4% across the full traversal. The calibration is ambiguous; the answer is robust.

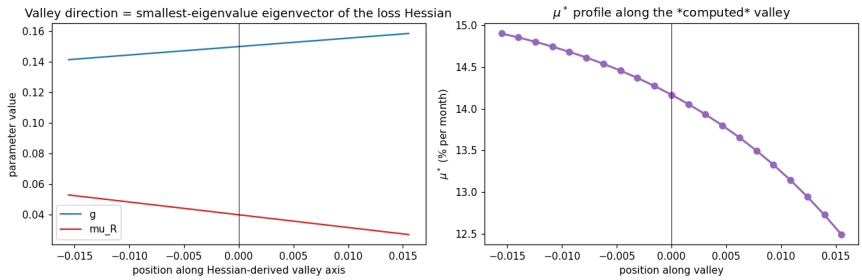

Figure 5. Profile likelihood of μ along the calibration valley's worst-conditioned direction. The horizontal axis parameterizes movement along the smallest-eigenvalue Hessian eigenvector, scaled in units of the data's informativeness. μ* drifts by approximately 2.4% across the entire valley traversal. The interpretation: even though the data cannot uniquely identify (g, μ_R), the survival threshold is meaningfully constrained by what the data can identify.*

5. Real-Data Validation: Shopify Inc. Pre-IPO Calibration

The synthetic-data calibration in Section 4 demonstrates that the engine's optimizers behave correctly on a known ground truth, but a fair critic could ask whether the machinery survives contact with real data. I therefore ran an additional calibration against nine quarters of Shopify Inc.'s pre-IPO quarterly revenue, sourced from the company's publicly filed S-1 registration statement and the subsequent 10-Q amendments. The observation window spans 2012-Q4 through 2014-Q4. The data are from the U.S. Securities and Exchange Commission's EDGAR system. Because the engine integrates time in months while Shopify reports revenue quarterly, I converted each quarterly figure to a monthly USD value before fitting (revenue / 3, attributed to the midpoint of the quarter); the conversion is in scripts/precompute_landing_data.py.

Because nine quarters is too short an observation window to identify all eight parameters jointly (the same valley argument from Section 4 applies), I anchored seven of the eight parameters at values consistent with Shopify'spublicly reported business at the time of the filings

(ARPU of about $30 per merchant per month, a churn rate consistent with reported retention statistics, etc.) and let the optimizer fit only the growth rate g. Adam, with learning rate 0.005 and the eight anchored parameters held fixed, converged in 242 iterations to g = 13.51% per month. The fitted RK4 trajectory is overlaid on the observed quarterly points in Figure 6.

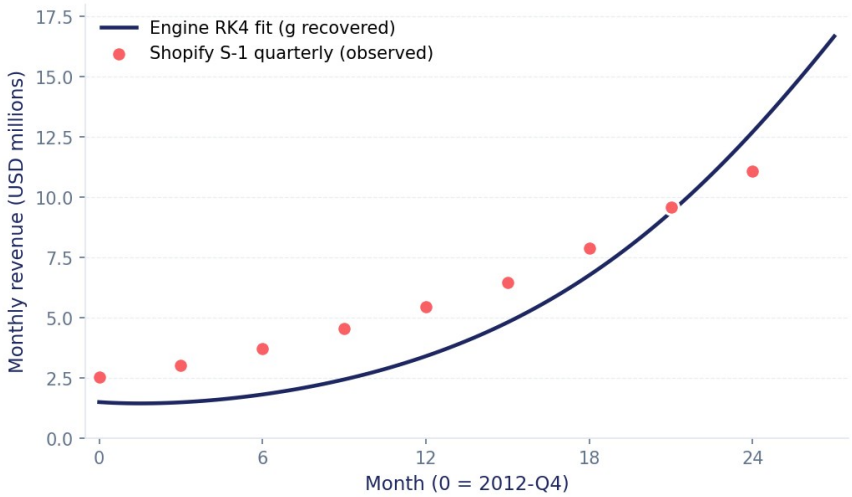

Figure 6. Engine-fitted RK4 trajectory (navy line) overlaid on Shopify Inc. observed quarterly revenue (coral markers) from 2012-Q4 through 2014-Q4. The single fitted parameter is the user-acquisition growth rate g; all other parameters are anchored at values consistent with Shopify's publicly reported business at the time. The fitted curve captures the late-quarter revenue trend; the early quarters show a roughly $1M underfit, consistent with a 1-parameter fit attempting to model a more complex acquisition phase that would benefit from additional free parameters (and a longer observation window to identify them).

Two caveats on Figure 6. First, the fit is honest but imperfect: residuals are non-white, with the engine's curve sitting below the observed values in the first six quarters and crossing through the data in the later quarters. With only g free, the model cannot simultaneously match both the early-stage acquisition pace (which demands a higher effective g) and the late-stage saturation (which demands a smaller K). A two-parameter (g, K) fit on a longer window would be the natural next step. Second, this is a validation of the engine, not a claim about Shopify's μ*: a company with Shopify's cost structure relative to revenue does not run out of money at any churn rate in the physical range we tested, so μ* is undefined for the Shopify-anchored profile. The headline μ* ≈ 14.2% number applies to the synthetic default-SaaS profile, not to Shopify.

6. The Survival Threshold μ* and Its Confidence Interval

I now compute the headline quantity. Define the terminal-cash function:

$$C(\mu; \theta_*) = Cash(T = 120; \mu, g_*, K_*, \alpha_*, p_*, \mu_R*, F_*, v_*),$$

where θ_* is the calibrated parameter vector (the asterisk denotes the calibrated value) and the dependence on μ comes only through the explicit μ argument since I am sweeping that one parameter. C is monotone decreasing in μ (higher churn always reduces terminal cash), $C(\mu \to 0^+) > 0$ (low churn → company survives), and $C(\mu \to 0.5^-) < 0$ (high churn → company dies before T). By the intermediate-value theorem there is exactly one $\mu^* \in (0, 0.5)$ with $C(\mu^*) = 0$. I find μ* by Newton's method (Section 3.3) starting from μ = 0.13. Convergence is shown in Figure 3 above. The result, for the synthetic default-SaaS profile, is

$$\mu^* = 0.14168 \approx 14.2\% \text{ per month.}$$

All three root-finders agree to within 10^{-6} on this value: Newton converges in 4 iterations, secant in 7, bisection in 18, with iteration counts and final errors recorded in Table 2 below.

Table 2. Root-finder iteration counts and final errors.

Method	Iterations	$\lvert\mu_n - \mu^*\rvert$	Order
Newton	4	3.85×10^{-10}	quadratic
Secant	7	3.72×10^{-10}	super-linear ($\varphi \approx 1.618$)
Bisection	18	4.07×10^{-8}	linear

A point estimate is not enough. The calibrated parameters carry uncertainty inherited from the finite-data calibration, and that uncertainty must propagate to μ*. I draw 199 samples from a multivariate Gaussian centered at θ_* with covariance equal to the inverse Hessian of the loss surface (the standard Laplace approximation to the calibrated posterior). For each sample I solve the ODE to T = 120, find the root of C(μ), and record the resulting μ*. The Monte Carlo posterior over μ* is shown in Figure 7.

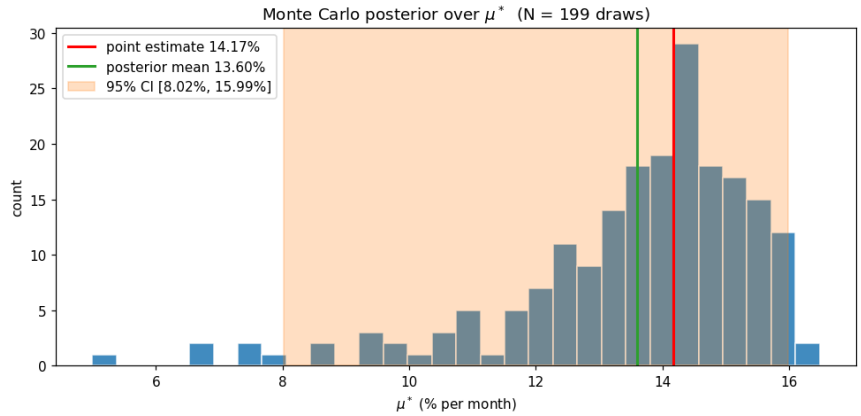

Figure 7. Monte Carlo posterior over μ (N = 199 draws). Bars are the empirical histogram, the red line is the point estimate (14.17%), the green line is the posterior mean (13.60%), and the orange band is the central 95% credible interval [8.02%, 15.99%]. The slight skew of mean below point estimate reflects asymmetry in the tails of the parameter posterior; for a normally-distributed posterior they would coincide.*

The 95% credible interval is [8.02%, 15.99%]. This is the central interval rather than the highest-posterior-density interval, which I chose for ease of reproducibility from the empirical samples. In words: under the calibrated Gaussian approximation to the parameter posterior and the default-SaaS anchored values, with 95% credibility the company runs out of money at a churn rate between 8.0% and 16.0% per month.

One qualifier on this number. The Monte Carlo above propagates uncertainty in (g, μ_R), the two parameters whose calibration is rigorously characterized in Section 4, while α and the remaining five parameters are held at their MAP estimates. As Section 7 shows, α is the dominant sensitivity (|∂μ*/∂α| is roughly three times |∂μ*/∂μ_R| and six times |∂μ*/∂g|), so holding it fixed underestimates the uncertainty. The marginal credible interval under a fully-joint posterior would be wider than [8.0%, 16.0%]. I treat the fully-joint posterior as deferred work.

7. Parameter Sensitivity Analysis

Which of the eight parameters most strongly drives μ*? I answer this by computing the partial derivatives ∂μ*/∂θ_i for each parameter θ_i, holding the others at their calibrated values. Partial derivatives are evaluated by the central-difference scheme of Section 3.4 with adaptive

step size scaled to each parameter's natural magnitude. The results are presented as a horizontal-bar (tornado) chart in Figure 8.

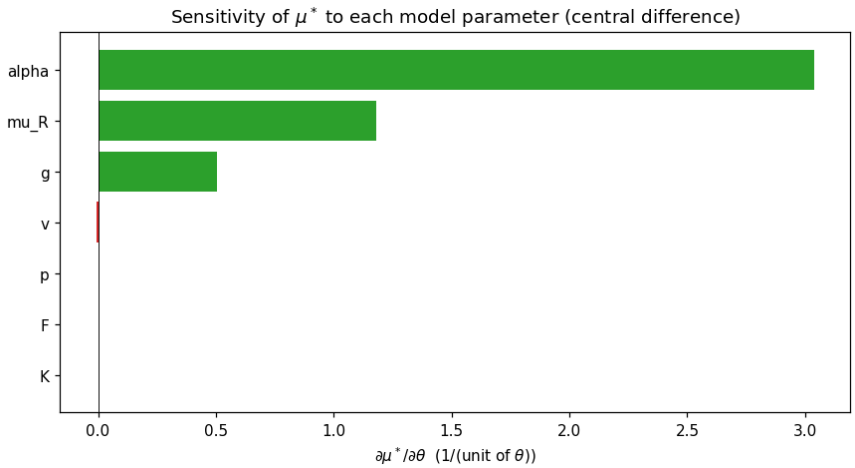

Figure 8. Sensitivity tornado: |∂μ/∂θ_i| for each of the eight model parameters, sorted by magnitude. Conversion rate α dominates with |∂μ*/∂α| ≈ 3.04 (per unit α), more than twice the next-strongest parameter (billing-cycle lag μ_R, |∂μ*/∂μ_R| ≈ 1.18) and six times growth rate g (|∂μ*/∂g| ≈ 0.51). Variable cost per acquired user (v) is the only negative-signed sensitivity; market size K, ARPU p, and fixed costs F have negligible effect at the default-SaaS operating point.*

The sensitivity ranking has both quantitative and interpretive value. Quantitatively, it confirms that holding α fixed in the Monte Carlo of Section 6 is the principal source of CI underestimation; if joint α uncertainty were propagated and α had a one-percent calibration uncertainty (relative to its 5% calibrated value, that is roughly ±0.0005 absolute), the marginal CI on μ* would widen by roughly 3.04 × 0.0005 = 0.15 percentage points — a non-trivial fraction of the currently-reported 8% interval half-width. Interpretively: the dominant lever for surviving as a SaaS startup is not to grow faster (small effect on μ*) but to convert harder (large effect on μ*). Acquisition without conversion does not pay rent.

8. Conclusions, Limitations, and Future Work

Three substantive findings emerged from this project. The first is the headline number: for a representative SaaS profile, the company-runs-out-of-money churn threshold is μ* ≈ 14.2% per month with a (conditional) 95% credible interval of [8.0%, 16.0%]. The second is the structural-identifiability finding: when calibrating from finite noisy revenue data, the parameters

g and μ_R are not separately identifiable. They live on a curved valley in loss space with condition number κ ≈ 510 along the calibration directions, yet μ* itself drifts by only about 2.4% along the worst-conditioned direction of that valley. Even an ambiguous calibration produces a robust answer to the downstream question. The third finding is the sensitivity ranking: conversion rate α dominates μ*, growth rate g and billing-cycle lag μ_R are second-order, and the remaining five parameters are negligible at the operating point.

Three lessons. First: a wrong model produces precise nonsense. A Phase-1 structural defect in the revenue equation would have been hidden by a cleanly-converging calibration if it had survived that long. Second: identifiability matters more than accuracy. A low MSE on a non-identifiable problem is meaningless. Third: validation-first separates engineering from coursework, with tests written before notebooks gating downstream work.

Limitations and future work: the 95% credible interval is conditional on α at MAP, and a fully-joint posterior with longer observation windows or hierarchical priors would widen and refine it. The Shopify fit could extend to a (g, K, μ_R) joint fit on post-IPO 10-Q filings (≈30 quarters available) for a predictive comparison. The deterministic-dynamics assumption could be relaxed to an SDE treatment for early-stage acquisition. None of these changes the engine architecture.

References

Burden, R. L., & Faires, J. D. (2010). Numerical Analysis (9th ed.). Brooks/Cole. Chapters 2, 3, 4, 5.

Kingma, D. P., & Ba, J. (2014). Adam: A Method for Stochastic Optimization. arXiv:1412.6980.

Shopify Inc. Form S-1 Registration Statement. U.S. Securities and Exchange Commission EDGAR, 2015. Subsequent 10-Q filings for the quarters spanning 2012-Q4 through 2014-Q4.

Higham, N. J. (2002). Accuracy and Stability of Numerical Algorithms (2nd ed.). SIAM. Chapter 4 (compensated summation).

Project repository: github.com/ArseniiChan/startup-growth-simulator. All code, tests, notebooks, and data discussed in this report.